Visual Studio LIVE! | Las Vegas
EXPERT SOLUTIONS FOR ENTERPRISE DEVELOPERS

Hardening CI/CD

Essential Strategies to Mitigate Security Risks

Ken Muse,
Staff DevOps Architect
ODST Team, GitHub

Level: Intermediate

Session Survey

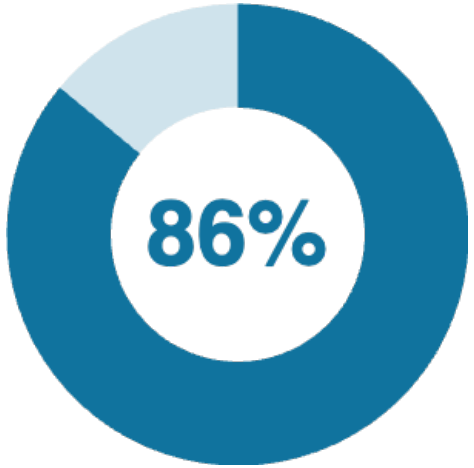
- Your feedback is very important to us
- Please take a moment to complete the session survey found in the mobile app
- Use the QR code or search for “Converge360 Events” in your app store
- Find this session on the Agenda tab
- Click “Session Evaluation”
- Thank you!

Today's journey

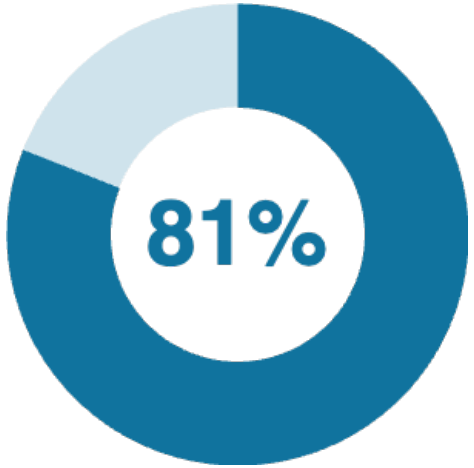
- Understanding the threats
- Case studies (with solutions)
- What about AI?

It's only a dev environment

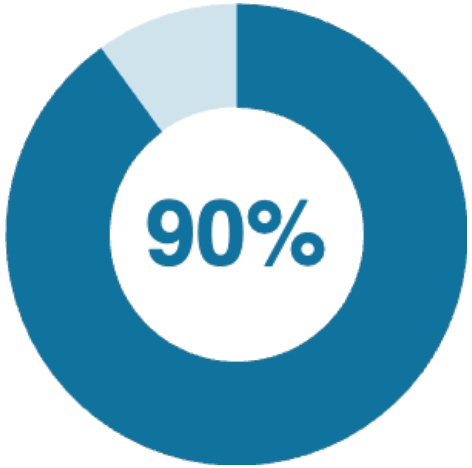
They target your systems for a reason



Organizations with repository exposed secrets



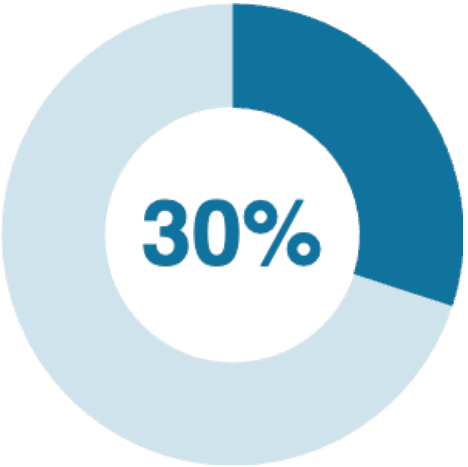
Code with critical vulnerabilities



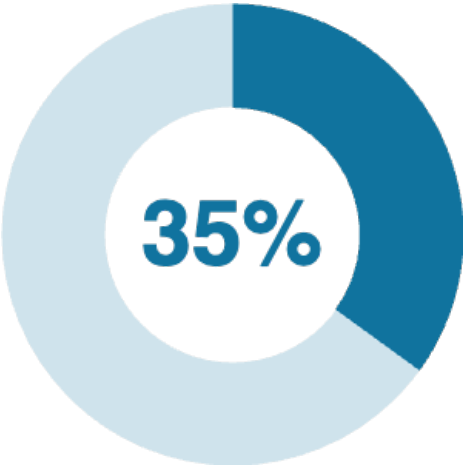
Packages 4 years or 10 versions behind current

Sources:
Wiz State of Code Security (2025)
Black Duck Open Source Security & Risk Analysis (2025)
Endor Labs Dependency Lifecycle Report (2023)

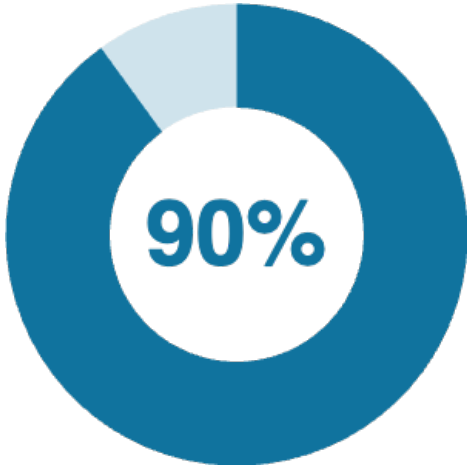
Your software is working against you



Breached with dependencies



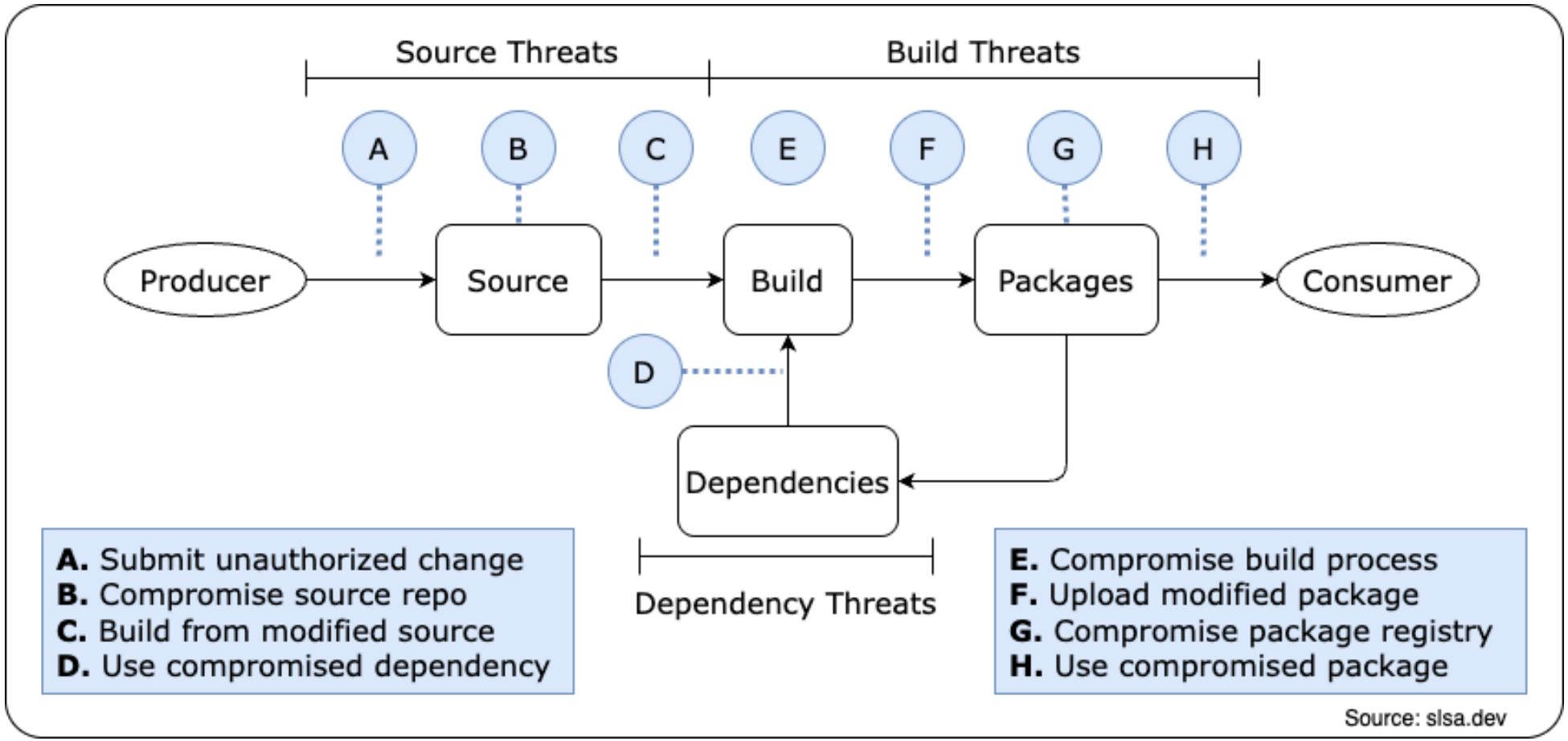
Enterprises with vulnerable non-ephemeral runners



NPM packages with vulnerable transitive dependencies

Sources:
Wiz State of Code Security (2025)
Verizon DBIR (2025)
Endor Labs Dependency Lifecycle Report (2023)

CI/CD threat model



How many of you regularly perform code reviews?

What's wrong with this code?

```
env:
  NPM_AUTH_TOKEN: ${{ secrets.NPM_AUTH_TOKEN }}

jobs:
  build:
  runs-on: ubuntu-latest
  steps:
  - name: Checkout
    uses: actions/checkout@v4
  - name: Setup Node.js
    run: curl -SsL https://npmjs.org/install.sh | bash
  - name: Install dependencies
    run: npm install
  - name: Validation message
    run: echo "Validating branch ${{ github.ref_name }}"
```

SolarWinds Orion (2020)

- More than 30,000 organizations affected
- Malicious code inserted before signing
- May have had up to 14 months of access before public report

Defensive measures

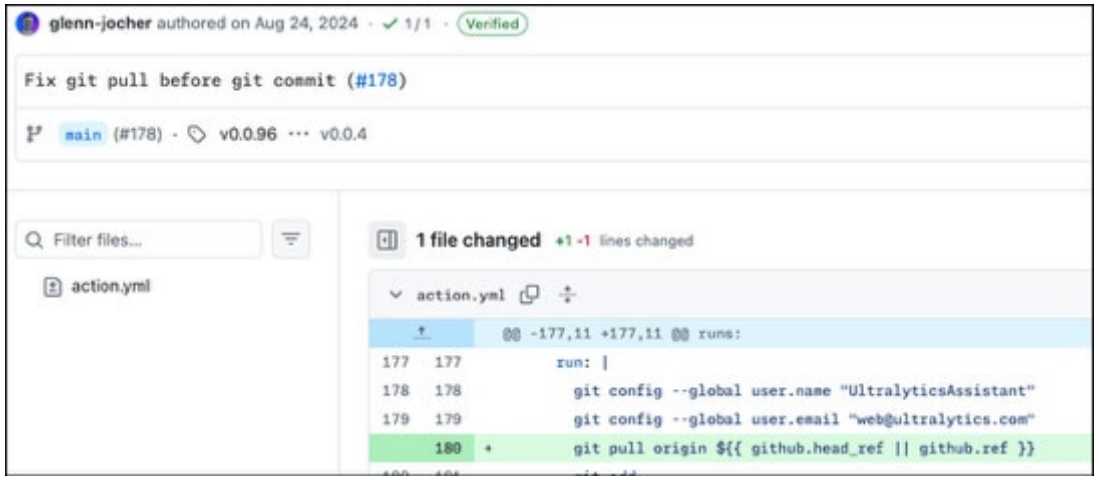
- Ephemeral build environments
- Runners on isolated networks by default
- Monitor for anomalous source file mutations

What's wrong with this workflow?

```
on:
  pull_request_target:
    branches: [main]
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - run: |
          git pull origin ${{ github.head_ref || github.ref }}
          git add .
          if ! git diff --stage --quiet; then
```

Ultralytics (2024/2025)

- Injection attack via branch name
- Cryptominer, exfuiltrated secrets, and poisoned build caches
- Used `pull_request_target` to access secrets



tj-actions/changed-files (2025)

- GitHub Actions exploit
- Initially used `pull_request_target` to steal tokens
- Compromised workflows that relied on floating tags
- Exfiltrated secrets to workflow logs

IAmRoot (2023)

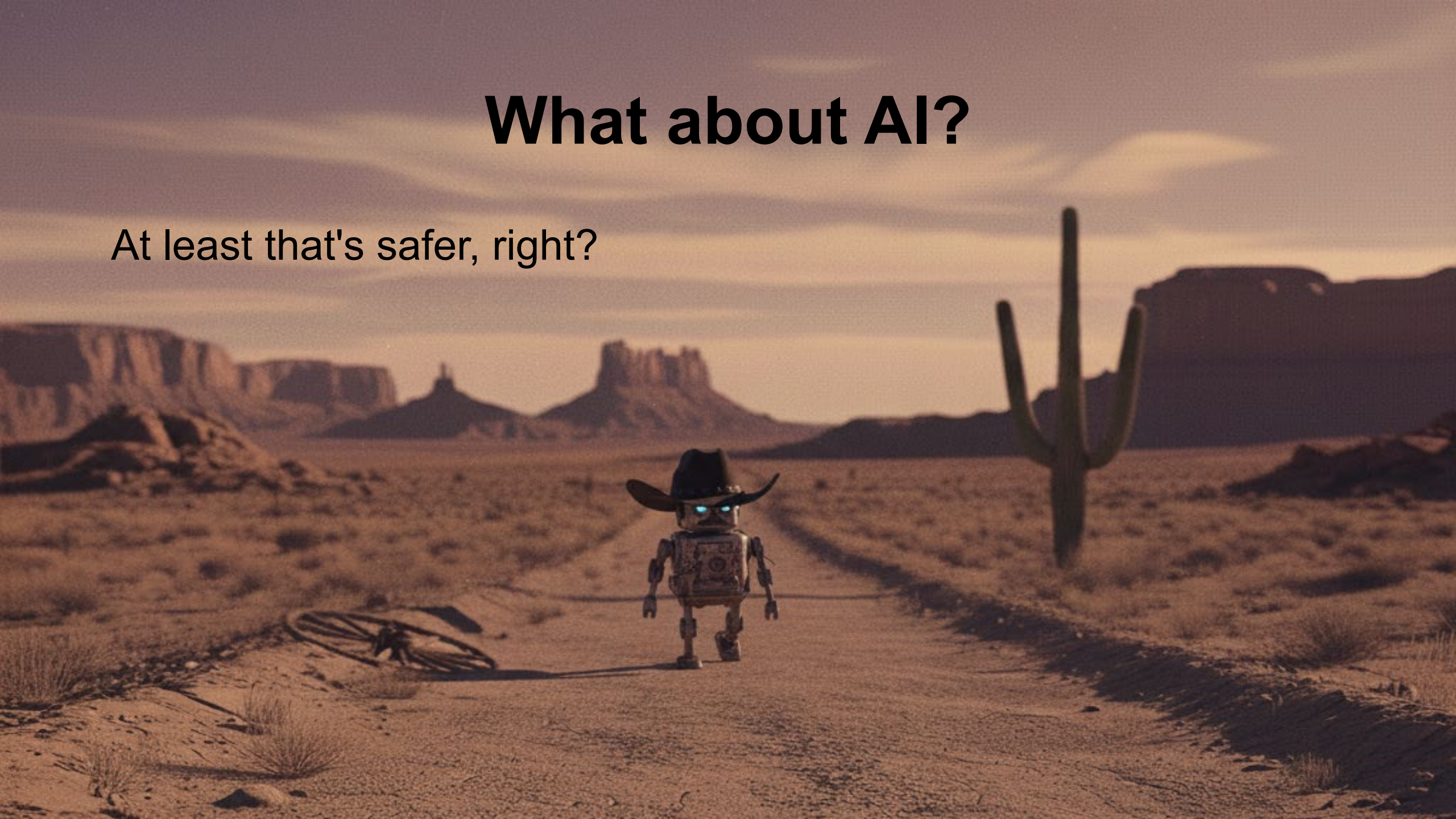
- NuGet packages contributed to the build process
- Used MSBuild to execute arbitrary code during restore
- Installed a remote access trojan
- Lasted almost 3 months

Shai-Hulud

- Malicious NPM postInstall
- Made private repositories public
- Exfiltrated secrets
- Compromised and replaced packages to spread

What about AI?

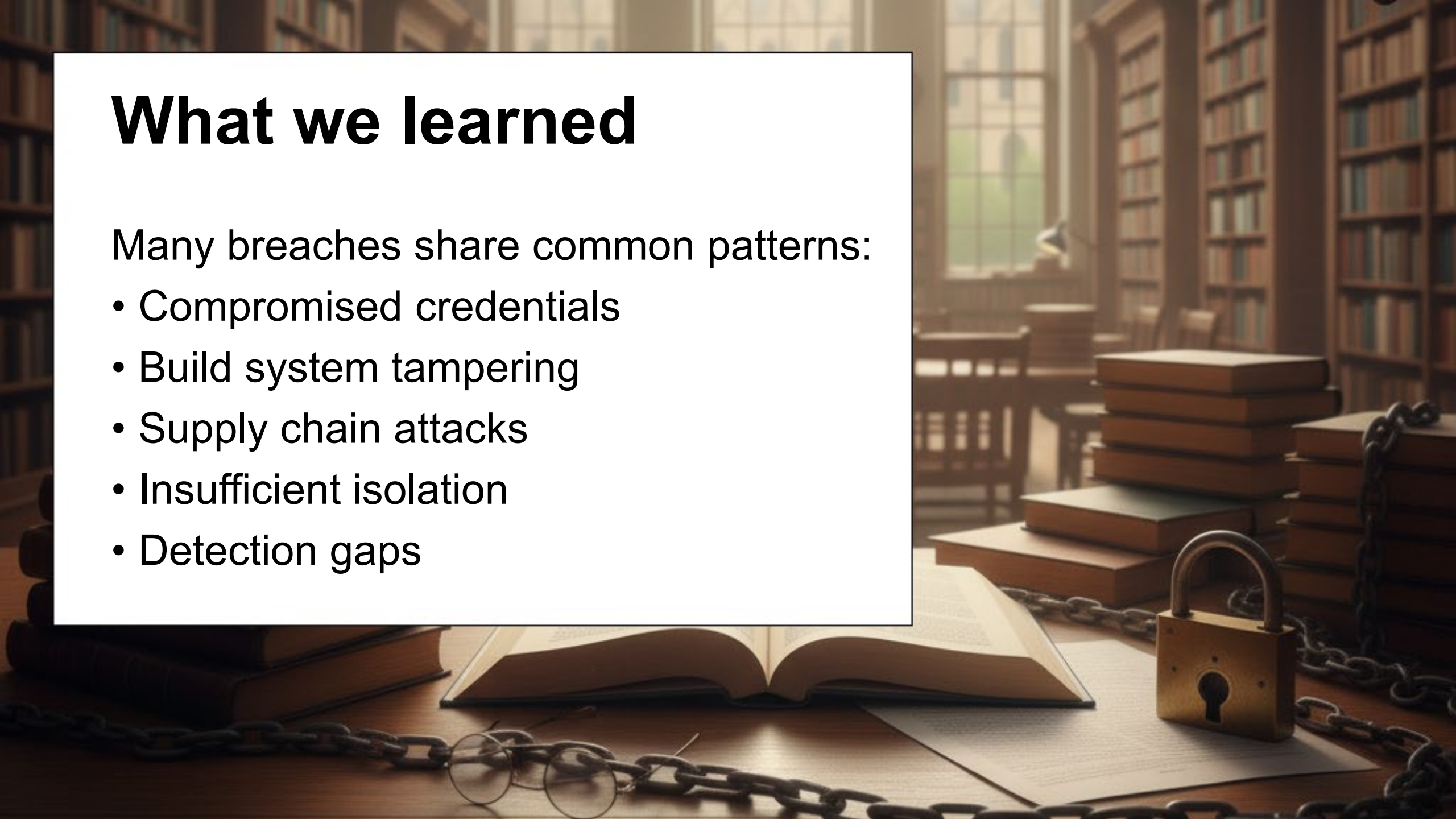
At least that's safer, right?



What we learned

Many breaches share common patterns:

- Compromised credentials
- Build system tampering
- Supply chain attacks
- Insufficient isolation
- Detection gaps



Cybersecurity isn't a cost center. It's an investment in your future

Thank you

Don't forget your session evaluation!